

Aplicación móvil con IA para emprendedores

MVP funcional · 27/08/2026 → 23/09/2026

Objetivo central: llegar a una demo de punta a punta donde una persona pueda registrarse, crear su emprendimiento, cargar información, ejecutar un análisis con IA, ver una estimación, pasar por un paywall/suscripción de prueba y entrar a un dashboard funcional.

Integrante	Rol principal
Ayrton	Tech Lead · Backend + IA
Lu	UI/UX + Frontend
Matu	Producto · Investigación · QA

Documento de trabajo interno · versión detallada

1. Qué es el proyecto

La aplicación está pensada como un asistente integral para personas que tienen una idea de negocio, están empezando un emprendimiento o ya llevan tiempo trabajando en uno. El usuario crea un “workspace” o espacio propio para su emprendimiento y allí concentra información, métricas, archivos y análisis.

La propuesta combina una aplicación móvil con un backend, base de datos, almacenamiento y servicios de IA. La IA no debe trabajar de manera genérica: debe utilizar exclusivamente el contexto del workspace activo, evitando mezclar datos entre emprendimientos distintos.

Flujo ideal de demostración del MVP

- 1. Crear una cuenta e iniciar sesión.
- 2. Crear un emprendimiento/workspace.
- 3. Cargar información básica del negocio.
- 4. Ejecutar un análisis inicial con IA.
- 5. Mostrar una estimación o proyección de crecimiento, presentada como escenario y no como garantía.
- 6. Mostrar el paywall o comparación de planes.
- 7. Activar una suscripción/pago de prueba.
- 8. Entrar al dashboard y ver información persistida.

Regla de alcance: todo lo que no sea necesario para completar este recorrido puede quedar para después del 23/09. El MVP debe ser demostrable; no necesita ser la versión definitiva del producto.

2. Arquitectura recomendada para el trabajo

El plan original define la cadena general: aplicación móvil → backend/API → base de datos → almacenamiento → IA. Para el equipo, una implementación coherente sería la siguiente:

Capa	Responsabilidad	Responsable principal
App móvil (Expo/React Native)	Pantallas, navegación, formularios, dashboard y consumo de la API.	Lu
Backend/API	Reglas de negocio, autenticación, permisos, endpoints y conexión con servicios.	Ayrton
Base de datos	Usuarios, workspaces, archivos, métricas, análisis y planes.	Ayrton
Storage	Guardar archivos e imágenes asociados a cada workspace.	Ayrton
IA + recuperación de información	Análisis, uso del contexto del workspace, búsqueda e interpretación	Ayrton

de datos.

El armado final de la APK queda fuera de las prioridades individuales de estas semanas porque el equipo lo realizará en el colegio cuando se enseñe el proceso. Hasta entonces, el foco debe estar en que la app funcione correctamente en desarrollo y se conecte al backend.

3. División del trabajo del equipo

Ayrton — Tech Lead / Backend + IA

- Definir arquitectura y estructura de repositorio.
- Implementar backend/API y conexión con la base de datos.
- Resolver autenticación, usuarios, sesiones y permisos.
- Diseñar y programar workspaces, almacenamiento y límites de planes.
- Crear los endpoints que alimentan el dashboard.
- Integrar la IA y asegurar contexto aislado por workspace.
- Implementar la estrategia de investigación/búsqueda.
- Intentar integración con Instagram o, si la API bloquea el avance, trabajar con un dataset controlado.
- Implementar proyecciones como estimaciones, no promesas.
- Integrar las partes del sistema y resolver bugs técnicos críticos.

Lu — UI/UX + Frontend

- Definir identidad visual, tipografía, colores e iconografía.
- Diseñar wireframes y flujo de navegación.
- Diseñar login, registro, onboarding, creación de workspace, dashboard, análisis y paywall.
- Implementar las pantallas en Expo/React Native.
- Crear componentes reutilizables: botones, inputs, tarjetas, modales y navegación.
- Conectar las pantallas con la API a medida que los endpoints estén disponibles.
- Preparar estados de carga, error y contenido vacío.
- Hacer el pulido visual y adaptación a distintos tamaños de pantalla.

Matu — Producto / Investigación / QA

- Redactar especificación funcional del MVP y casos de uso.
- Investigar competidores, fuentes de datos y APIs necesarias.
- Definir métricas del dashboard y límites de planes.
- Preparar datos de prueba realistas.
- Definir qué debe responder la IA y crear ejemplos de entrada/salida.
- Probar cada flujo a medida que aparece.
- Registrar bugs con pasos de reproducción y prioridad.
- Hacer pruebas de regresión, casos límite y prueba general antes de la entrega.

Regla de coordinación: Lu no debería quedar bloqueada esperando al backend. Ayrton y Lu deben acordar desde el principio qué endpoints existirán y qué formato de datos devolverán, para que el frontend pueda avanzar con datos mockeados.

4. Plan personal de Ayrton — semana por semana

SEMANA 1 · 27/08 al 02/09 — Fundaciones

Objetivo de la semana: terminar con el esqueleto técnico listo. No hace falta IA todavía. Deben existir proyecto, arquitectura, backend, base de datos, modelo inicial y autenticación encaminada.

Paso a paso recomendado

Paso 1. Crear y ordenar el repositorio — Definir una estructura clara para app móvil y backend. Acordar ramas o una estrategia de trabajo sencilla y dejar un README con instrucciones para ejecutar cada parte.

Paso 2. Dejar Expo funcionando — Crear o verificar el proyecto React Native con Expo. La meta es que cualquier integrante pueda clonar, instalar dependencias y abrir la app sin depender de tu computadora.

Paso 3. Crear el backend mínimo — Levantar el servidor y agregar un endpoint de salud, por ejemplo GET /health. Probar desde la app que puede recibir una respuesta HTTP del backend.

Paso 4. Definir variables de entorno — Separar claves y configuraciones del código. No poner claves privadas de IA, base de datos o servicios dentro de la app móvil.

Paso 5. Diseñar el modelo de datos — Como mínimo contemplar usuarios, planes, workspaces, archivos, métricas e historial de análisis. Definir relaciones y qué datos pertenecen a cada workspace.

Paso 6. Conectar la base de datos — Crear la conexión desde el backend y hacer una primera lectura/escritura real.

Paso 7. Implementar autenticación — Registro, login y manejo de sesión. Verificar que un usuario no pueda acceder a recursos de otro.

Paso 8. Definir límites de planes desde código — Aunque el pago sea simulado, dejar configurables límites como cantidad de workspaces y almacenamiento.

Paso 9. Acordar contrato de API con Lu — Definir nombres de endpoints, campos que recibe cada formulario y formato de respuesta. Lu puede avanzar con mocks.

Paso 10. Cierre de semana — Hacer una prueba desde cero en otra máquina o instalación limpia y documentar qué funciona y qué queda pendiente.

Checklist de cierre de Semana 1

- ☐ Repo compartido y ejecutable.
- ☐ App Expo abre correctamente.

- ☐ Backend responde /health.
- ☐ App se comunica con backend.
- ☐ Base de datos conectada.
- ☐ Modelo de datos inicial definido.
- ☐ Registro/login funcionando o muy cerca de quedar cerrado.
- ☐ Separación de variables de entorno.
- ☐ Contrato inicial de API acordado con Lu.

SEMANA 2 · 03/09 al 09/09 — Usuarios, workspaces, storage y dashboard base

Objetivo de la semana: lograr persistencia real. El usuario debe poder entrar, crear su emprendimiento, volver más tarde y seguir viendo su información. También debe existir una base funcional para archivos y dashboard.

Paso 1. Cerrar autenticación — Resolver cualquier pendiente de sesión, expiración, validación y usuario actual.

Paso 2. CRUD de workspaces — Crear, listar, seleccionar, editar y eliminar lógicamente un emprendimiento. Cada operación debe validar que el workspace pertenece al usuario autenticado.

Paso 3. Aislamiento de datos — Probar explícitamente que el usuario A nunca puede leer, modificar ni analizar el workspace del usuario B.

Paso 4. Storage — Permitir carga de archivos y asociarlos al workspace correspondiente. Aplicar validaciones de tipo/tamaño según el alcance definido.

Paso 5. Límites por plan — Impedir nuevas cargas o nuevos workspaces cuando se alcance el límite configurado.

Paso 6. Endpoints del dashboard — Crear endpoints para métricas básicas. Al principio pueden devolver datos de prueba o datos ingresados manualmente.

Paso 7. Dashboard funcional con datos de prueba — Trabajar con Lu para que el frontend ya dibuje tarjetas y métricas consumiendo el backend.

Paso 8. Preparar la capa de IA — Dejar definida la forma en que el backend recuperará datos del workspace para convertirlos en contexto en la semana siguiente.

Paso 9. Pruebas con Matu — Entregarle flujos concretos para romper: usuario nuevo, workspace vacío, límite de plan, archivo inválido, intento de acceso ajeno.

Hito del 09/09

Base funcional: usuarios, workspaces, almacenamiento y dashboard base funcionando. Si esto no está estable, no conviene saltar a funciones avanzadas de IA.

SEMANA 3 · 10/09 al 16/09 — IA, análisis, Instagram y paywall

Objetivo de la semana: conectar el corazón “inteligente” de la app y cerrar el flujo completo hasta el dashboard. Esta es la semana de mayor riesgo técnico, por lo que conviene implementar de menor a mayor complejidad.

Paso 1. Integrar el proveedor de IA — Hacer una prueba simple desde el backend. La clave debe permanecer exclusivamente del lado servidor.

Paso 2. Crear el contexto por workspace — Construir una función que reúna descripción, rubro, objetivos, métricas, archivos procesados y análisis previos del workspace activo.

Paso 3. Primera respuesta contextual — Hacer que la IA responda una pregunta utilizando solo ese contexto. Probar con dos workspaces distintos y verificar que no se mezclen.

Paso 4. Análisis inicial estructurado — Definir con Matu una salida estable: viabilidad, demanda, competencia, riesgos, oportunidades, recomendaciones y explicación. Conviene pedir una respuesta estructurada para que el frontend pueda mostrarla en tarjetas.

Paso 5. Estrategia de búsqueda/investigación — Agregar la recuperación de información externa definida por el equipo. Si no hay una integración confiable, reducir el alcance antes que bloquear la demo.

Paso 6. Persistir análisis — Guardar fecha, entrada, resultado y workspace. Así la app puede mostrar historial y la IA puede aprovechar análisis previos.

Paso 7. Proyección de crecimiento — Calcular o generar escenarios a 1 y 3 meses usando los datos disponibles. Mostrar rangos o escenarios; nunca presentarlos como garantía.

Paso 8. Instagram — Intentar la integración real solo si los permisos y la API lo permiten dentro del tiempo disponible. Si se complica, activar un dataset controlado para demostrar análisis de métricas.

Paso 9. Paywall y suscripción simulada — Exponer endpoints/estado de plan para que, después de la estimación, la app muestre planes y permita activar una suscripción de prueba.

Paso 10. Flujo completo — Probar onboarding → análisis → estimación → paywall → suscripción → dashboard sin modificar datos manualmente en la base durante la demo.

Plan B obligatorio para Instagram

La integración con Meta puede requerir permisos, configuración y revisión que excedan el tiempo del MVP. Por eso, el plan original permite usar los datos que realmente puedan obtenerse o un conjunto de datos controlado. El proyecto no debe depender de que Instagram real funcione para aprobar.

SEMANA 4 · 17/09 al 23/09 — Integración, estabilización y entrega

Objetivo de la semana: dejar de agregar funciones y convertir el sistema en algo estable. La meta interna es tener todo integrado el 20/09 y reservar los últimos días para pruebas, bugs y presentación.

Paso 1. Congelar funcionalidades — No agregar ideas nuevas salvo que sean imprescindibles para el recorrido de demo.

Paso 2. Integración total — Verificar app + backend + DB + storage + IA como un único sistema.

Paso 3. Persistencia y permisos — Cerrar sesión, volver a entrar, cambiar de workspace y validar que todos los datos sigan correctos.

Paso 4. Rendimiento — Identificar llamadas lentas, agregar mensajes/estados de carga y evitar que la interfaz parezca congelada.

Paso 5. Bugs críticos — Resolver primero bloqueantes y críticos. Los cosméticos quedan para después si el tiempo no alcanza.

Paso 6. Entorno de demostración — Preparar una cuenta y datos de prueba que permitan mostrar el producto sin depender de improvisar información durante la exposición.

Paso 7. Prueba de regresión con Matu — Repetir todos los flujos principales después de cada corrección importante.

Paso 8. Ensayo completo — Una persona que no programó el backend debe poder completar la demo sin intervención manual tuya.

Paso 9. APK en clase — Cuando el proceso se enseñe en el colegio, realizar el build de la APK entre todos. Hasta ese momento no debe bloquear ninguna semana del plan.

5. Contrato mínimo de API que conviene definir temprano

Esta sección es una recomendación técnica para evitar bloqueos entre frontend y backend. Los nombres exactos pueden cambiar; lo importante es acordar qué operaciones existirán y qué información intercambian.

Endpoint	Uso
GET /health	Comprobar que el backend está activo.
POST /auth/register	Crear usuario.
POST /auth/login	Iniciar sesión.
GET /me	Obtener usuario actual.
GET /workspaces	Listar emprendimientos del usuario.
POST /workspaces	Crear emprendimiento.
GET /workspaces/:id	Leer un workspace propio.
PATCH /workspaces/:id	Editar información del emprendimiento.
DELETE /workspaces/:id	Eliminación lógica.

POST /workspaces/:id/files	Subir archivo al workspace.
GET /workspaces/:id/dashboard	Obtener métricas del dashboard.
POST /workspaces/:id/analyze	Ejecutar análisis con IA.
GET /workspaces/:id/analyses	Historial de análisis.
POST /subscription/activate-demo	Activar plan de prueba/simulado.

6. Modelo de datos inicial

El plan exige usuarios, planes, workspaces, archivos, métricas e historial de análisis. Una estructura inicial puede ser:

- users: identidad, email, datos de autenticación/relación con el proveedor de auth.
- plans: nombre del plan, límites de workspaces, almacenamiento y funciones habilitadas.
- workspaces: owner_user_id, nombre, descripción, etapa, rubro, público objetivo, objetivos e Instagram opcional.
- files: workspace_id, nombre, tipo, tamaño, ubicación de storage y metadatos.
- metrics: workspace_id, tipo de métrica, valor, fecha y fuente.
- analyses: workspace_id, fecha, entrada, resultado estructurado, fuentes/contexto usado.
- subscriptions: user_id, plan_id, estado y fechas; en el MVP puede ser una activación simulada.

Regla de seguridad lógica: toda tabla relacionada con un emprendimiento debe poder rastrearse hasta el workspace y, desde allí, hasta el usuario propietario.

7. Cómo coordinarse para no bloquearse

- Reunión corta al inicio de cada semana: objetivo, dependencias y qué debe quedar demostrable el último día.
- Ayrton publica endpoints y ejemplos de respuesta; Lu desarrolla con mocks si el endpoint todavía no existe.
- Matu prueba desde que aparece la primera funcionalidad, no solamente al final.
- Cada bug debe incluir: pasos para reproducir, resultado esperado, resultado real y prioridad.
- No mezclar features nuevas con correcciones críticas durante los últimos días.
- Usar datos de prueba compartidos para que los tres vean el mismo comportamiento.

8. Criterios de éxito por fecha

Fecha	Hito	Debe estar listo
02/09	Fundaciones	Arquitectura, diseño base, especificación, auth y modelo de datos

		definidos.
09/09	Base funcional	Usuarios, workspaces, almacenamiento y dashboard base funcionando.
16/09	Feature-complete	IA, análisis, estimación, paywall y dashboard conectados en un flujo completo.
20/09	Meta interna	Todo integrado; desde aquí solo bugs, pruebas y pulido.
23/09	Entrega	MVP funcional y demostrable de punta a punta.

9. Orden de prioridad personal de Ayrton

Cuando tengas dudas sobre qué hacer primero, usá este orden. No avances al bloque siguiente si el anterior está roto de manera crítica.

#	Bloque	Condición para avanzar
1	Proyecto ejecutable	Expo + backend arrancan y el equipo puede correrlos.
2	Conectividad	La app puede hablar con el backend.
3	Persistencia	El backend lee y escribe en la base de datos.
4	Identidad	Registro, login, sesión y usuario actual.
5	Aislamiento	Un usuario solo accede a sus workspaces.
6	Workspaces	CRUD y persistencia de emprendimientos.
7	Archivos y límites	Storage y reglas de plan.
8	Dashboard	Endpoints y métricas básicas.
9	IA contextual	Análisis usando datos del workspace.
10	Búsqueda/proyección	Investigación y estimaciones.
11	Instagram	Real si es viable; dataset controlado si no.
12	Paywall	Plan/suscripción de demo.
13	Integración	Flujo completo y estable.
14	APK	Se realiza entre todos en el colegio

cuando lo enseñen.

10. “Definition of Done” para cada funcionalidad

Una tarea no debería considerarse terminada solo porque “funciona en tu computadora”. Antes de marcarla como lista, comprobar:

- ☐ Funciona desde la app, no solo desde Postman o una herramienta de backend.
- ☐ Valida errores comunes y muestra una respuesta clara.
- ☐ Respetar autenticación y permisos.
- ☐ Los datos quedan persistidos si corresponde.
- ☐ Matu puede reproducir el flujo con pasos documentados.
- ☐ Lu conoce el formato de datos si la función afecta al frontend.
- ☐ No rompe el recorrido principal del MVP.

11. Guion sugerido para la demo del 23/09

9. Abrir la app y mostrar login/registro.
10. Crear un usuario de demostración.
11. Completar onboarding y crear un emprendimiento.
12. Cargar información o un archivo de ejemplo.
13. Ejecutar el análisis con IA y explicar que usa el contexto del workspace.
14. Mostrar una estimación/proyección y aclarar que es un escenario, no una garantía.
15. Mostrar el paywall y activar una suscripción de prueba.
16. Abrir el dashboard y enseñar métricas.
17. Cerrar sesión, volver a entrar y comprobar persistencia.
18. Si hay tiempo, mostrar un segundo workspace para demostrar separación de datos.

Criterio de oro: si otra persona puede realizar esta demo completa sin que Ayrton tenga que tocar la base de datos, ejecutar comandos manuales o corregir datos durante la presentación, el MVP cumple su objetivo.

12. Riesgos que NO deben frenar el proyecto

Riesgo	Decisión para el MVP
Instagram real	Usar dataset controlado si permisos o API se complican.
Pago real	Mantener pago/suscripción simulada para el MVP.
Búsqueda web compleja	Reducir fuentes o utilizar una estrategia más simple y explicable.
Diseño perfecto	Priorizar flujo funcional; pulido visual en la última semana.

Funciones nuevas

Mandarlas a backlog si no forman parte del recorrido de demo.

APK

No dedicar tiempo individual ahora: se hará con el equipo en clase.

13. Checklist rápida de Ayrton

- Semana 1: repo + Expo + backend + DB + auth + modelo de datos + contrato API.
- Semana 2: workspaces + aislamiento + storage + límites + dashboard/API.
- Semana 3: IA contextual + análisis + búsqueda + proyección + Instagram/plan B + paywall.
- Semana 4: integración + permisos + persistencia + rendimiento + bugs + demo.
- En clase: build final de APK con todo el equipo.